

Encrypted Case Container — Options, Setup, and Trade-offs

2026-08-02 · Companion to the Operating Manual v1.3 and the Claude Code Handover

1. What this actually protects against

Worth being precise, because full-disk encryption and a container solve different problems and it is easy to assume one covers the other.

Threat	Full-disk encryption	Case container
Laptop stolen while off	Protects	Protects
Laptop stolen while running	Does not protect	Protects, if closed
Backup or clone of the disk	Protects	Protects
A process on the running machine reads the files	Does not protect	Protects, if closed
You forget the boundary at 11pm in week three	Does not protect	Protects structurally

Full-disk encryption is unlocked the moment you log in. From then on, every process running as your user can read every file, which includes anything you attach to the session. The container closes that window by making the data absent rather than merely permissioned. A file that is not mounted cannot be read by the Read tool, by grep, by a script, or by a confused agent — the path is simply empty.

Both are worth having. They are not alternatives.

2. The piece that makes it systemic

Encryption alone is still a procedure: remember to close the container before starting a session. Procedures fail. What converts it into a property of the system is a small wrapper that shadows the claude command and refuses to launch while the container is mounted.

```
$ cd "$KIT" && claude
```

BLOCKED: the case container is mounted at
~/translation-work/confidential-projects

Claude Code sends context to Anthropic's servers. Case material must not be reachable during a session.

Close it first: case-close
Then start from the kit directory (outside ~/translation-work)

The wrapper makes three checks, all verified working. It refuses if the container is mounted. It refuses if the data directory is not a mount but holds files anyway — which means case material is sitting unencrypted outside the container, and closing the container cannot hide it. And it refuses if the working directory sits inside the data path or is a parent of it, which is what happens if you start a session from your home directory. The three exit with distinct codes (3, 5 and 4) so a script can tell them apart.

Forgetting now produces a refusal instead of a silent exposure. That is the whole argument for doing this.

3. Options

3.1 LUKS file container — recommended

A single encrypted file, unlocked with a passphrase and mounted as a filesystem. Kernel-native on Linux, no third-party software, and the same mechanism Ubuntu uses for full-disk encryption. Best performance of the options here because encryption happens at the block layer.

Recommended because it is the strongest option that requires nothing you do not already have, and because it fails closed: if it is not unlocked, there is nothing there at all — not filenames, not sizes, not directory structure.

3.2 gocryptfs — if backups matter more

Encrypts file by file rather than as one block device. Grows dynamically with no fixed size, and because each file is separately encrypted, incremental backup works normally — changing one document syncs one file rather than a 40 GB image.

The trade is metadata leakage. An observer with access to the encrypted directory sees how many files exist, their approximate sizes, and the directory shape, even without reading content. For a document set whose very structure might be informative, that is a real if modest concession.

3.3 VeraCrypt — if the Windows machine needs access

Cross-platform, with a graphical interface. Worth it only if the same container must open on your Windows machine; otherwise it adds a third-party dependency and some performance cost for no benefit over LUKS.

3.4 External encrypted drive — strongest, least convenient

A LUKS-encrypted USB SSD that you physically unplug. Nothing beats the boundary of a disconnected cable, and it makes the container impossible to forget about because its absence is visible. The cost is that batch runs must complete while it is attached, and a multi-day run means a drive left plugged in for days — which erodes the advantage.

Reasonable hybrid: work on the internal container, and keep the encrypted external drive as the backup target.

4. Setup

Roughly fifteen minutes, most of it choosing a passphrase. The kit now includes five scripts that do the work.

4.1 If a plaintext work directory already exists

Move it aside first - ~/translate was the layout before the container, so a machine set up earlier may still have one. case-init refuses to run over an existing directory with contents.

```
mv ~/translate ~/translation-work/plain-backup
```

4.2 Create the container

```
cd "$KIT" # the kit, under Claude_Stuff/cli_projects/  
./bin/case-init 40G
```

This creates a sparse LUKS2 file at ~/.case/confidential.luks, formats it ext4, seeds the _shared/ layer with the glossary and prompt templates, backs up the LUKS header, and makes the bare mountpoint immutable so nothing can write to it while closed. Individual projects are created afterwards with tr-project --new.

Forty gigabytes is generous for several matters of roughly one hundred documents each plus their OCR output; the file is sparse, so it consumes only what is actually used. The model itself lives under the Ollama directory, not here.

Two things to do immediately after. Put the passphrase in your password manager — there is no recovery. And copy ~/.case/header.bak somewhere off this machine: without the header backup, corruption of the first few megabytes destroys the data even if the passphrase is correct.

4.3 Move the existing data in

```
case-open  
rsync -a ~/translation-work/plain-backup/ \  
~/translation-work/confidential-projects/kranj-2024/  
case-status # confirm the file counts look right  
  
# only once verified:  
rm -rf ~/translation-work/plain-backup
```

On an SSD, shred does not reliably overwrite in place because of wear levelling. Treat it as best-effort. The stronger guarantee comes from having had full-disk encryption on all along, which is why both layers matter.

4.4 Install the guard

This is the step that makes the whole arrangement structural rather than procedural. Do not skip it.

```
mkdir -p ~/.local/sbin
```

```

cp "$KIT"/bin/case-guard ~/.local/sbin/claude
chmod +x ~/.local/sbin/claude

# ~/.local/sbin must come first in PATH
echo 'export PATH="$HOME/.local/sbin:$PATH"' >> ~/.bashrc
source ~/.bashrc

# verify: this should launch normally
cd "$KIT" && claude --version

# verify the block: this should refuse
case-open && cd "$KIT" && claude # expect a refusal

```

The wrapper resolves the real binary by skipping itself in the PATH search, so it cannot recurse. If it cannot find the real claude it exits rather than failing open.

The PATH wrapper only covers the terminal. The desktop application is launched from a .desktop entry as /usr/bin/claude-desktop and runs its own bundled copy of Claude Code out of ~/.config/Claude/, so nothing on PATH is consulted and case-guard never sees it. That left one hole in the mutual exclusion, and it is the one that actually happened: opening the container and then starting the desktop app. case-guard-desktop closes it, installed through a user .desktop file that shadows the system one, so the launcher, the dock and the claude:// handler all go through it.

```

tools/install-desktop-guard.sh # writes the .desktop override

```

There is no terminal behind a GUI launch, so the refusal appears as a dialog rather than on stdout. Test it with case-guard-desktop --check, which prints the decision and shows nothing on screen.

Confirm both guards are installed, rather than assuming it. case-status now reports each one, and both must read installed. This exists because the assumption failed: the wrapper makes the boundary structural instead of remembered, but running the installer was itself remembered, and on this machine it was not done. The terminal guard was in place, the desktop guard was never installed, and Claude Desktop launched with the container mounted. Every document said the boundary was closed; nothing checked.

4.5 Daily use

```

case-status      # open or closed? safe to start a session?
tr-project      # which matter is active?
case-open       # unlock and mount
cd ~ && case-close # unmount and lock
case-close -f   # force, when a batch is still running

```

cd out first, and not for tidiness. A shell whose working directory is inside the container keeps the filesystem busy exactly as an open file does. The kernel makes no distinction, and -f does not help - it forces past processes holding files, not past a working directory. Closing from inside the project directory fails either way; leaving it first always works.

case-close reports which processes still hold files open rather than failing with a bare "target is busy", which during a multi-day batch is the difference between a useful message and a puzzle. With fingerprint sudo already configured, each of these is a touch of the reader.

Closing does not delete anything. Closing the container unmounts a filesystem; it does not empty one. Everything written during a session is still inside the container file, encrypted, and comes back exactly as it was at the next case-open. The mountpoint looks empty only because nothing is mounted there.

The corollary is that nothing ever leaves on its own. A matter that is finished, delivered and paid for still occupies the container, in full, until somebody removes it by hand — open the container and delete the project directory. That is deliberate: no script deletes client work. But it means retention is a decision you have to make rather than one the system makes for you, and material you no longer have a reason to hold is material you are still holding.

case-open

```
rm -rf ~/translation-work/confidential-projects/kranj-2024
```

Sparse file, so the container does not shrink on disk when a project is removed; the space is reused by the next one. Check what is there with case-status, which lists each project and its file counts.

5. Trade-offs, honestly

Upside	Downside
Data is absent, not merely permissioned, while closed	One more thing that can be left in the wrong state
Protects against a running-machine compromise, which FDE does not	Fixed size; growing it later is a multi-step operation
Makes the Claude Code boundary enforceable rather than remembered	Backup copies the whole image — no incremental sync
Kernel-native; no third-party software to trust or maintain	Header corruption without a backup means total loss
Negligible performance cost on modern hardware with AES-NI	Lost passphrase means total loss, with no recovery path
Demonstrates deliberate handling if the arrangement is ever questioned	A long batch run means the container is open for days anyway

5.1 The honest limitation

A multi-day batch run requires the container to be open for those days. During that window the protection against a running-machine threat is suspended, and only full-disk encryption remains. The container does not fix this and nothing else would either — the data has to be readable to be processed.

What it does fix is everything outside that window, which is most of the project: development, debugging, glossary work, document review, and every Claude Code session. Those are also where the realistic exposure lies, because they are the activities that involve attaching tools to the data.

6. What I verified and what I did not

Verified	Not verified here
LUKS2 container creation produces a valid header	The unlock and mount cycle — this sandbox has no device-mapper
The guard blocks when the container is mounted	Behavior against the real Claude Code binary
The guard blocks from inside the data directory	chattr +i on your ext4 root (should work; verify)
The guard blocks from a parent directory	Fingerprint sudo interaction with these scripts
The guard passes through cleanly when safe, without recursing	rsync of a real corpus into the container

Run case-init on a small throwaway container first — case-init 1G with a different CASE_IMG and CASE_MNT — and exercise open, close, and the guard before committing real data. This is a good first task for the Claude Code session, since none of it involves case material.

```
CASE_IMG=/tmp/trial.luks CASE_MNT=/tmp/trialmnt \  
"$KIT"/bin/case-init 1G  
CASE_IMG=/tmp/trial.luks CASE_MNT=/tmp/trialmnt case-open  
CASE_IMG=/tmp/trial.luks CASE_MNT=/tmp/trialmnt case-status  
CASE_IMG=/tmp/trial.luks CASE_MNT=/tmp/trialmnt case-close
```

7. Recommendation

Do it, with the LUKS file container, and install the guard at the same time. The container without the guard is worth having; the guard is what answers the question you actually asked, which was how to stop this depending on memory.

The setup cost is about fifteen minutes and the daily cost is two commands. Against that, the failure it prevents is one you would not notice happening.

If your Windows machine ever needs to open the same container, revisit VeraCrypt before creating this one — converting later means copying everything through plaintext.